GPAS: OPTIMIZER-AWARE TASK ALLOCATION FROM SINGLE-TASK TO MIXED UPDATES

Anonymous authors
Paper under double-blind review

ABSTRACT

Multi-task LLM post-training must choose both what objective to optimize and how to execute it. We make the number of tasks represented in an optimizer update, K, an explicit configuration: task-at-a-time training is $K = 1$ and a fully mixed update is $K = M$. Each selected task contributes the same counted unit—a fixed number of prompts and responses—rather than a fixed token budget. This keeps the objective independent of response length and lets measured time capture the real cost of long generations.

For any K, a Horvitz–Thompson correction gives an unbiased gradient of one stated weighted teacher-loss objective. A simple variance decomposition then prices the choice of K: task-selection noise decreases as more tasks enter an update and vanishes at the fully mixed endpoint, while within-task noise remains. Gradient-Preconditioned Adaptive Sampling (GPAS) chooses task inclusion probabilities from objective weights and AdamW-scaled gradient magnitudes; capped water filling covers every K. Cost-GPAS additionally uses the measured critical-path cost of each task unit. Moment-consistent AdamW provides one proposal-independent taskwise second-moment target, while its difference from conventional mixed-batch AdamW must be measured rather than assumed beneficial.

We specify a four-teacher MOPD protocol that compares the two endpoints under a common per-response objective. It uses one matched run per method, bounded-age probe batches for online scale estimation, and places the warm-start headroom before any end-to-end claim. The system accounting separates teacher residency from switching cost: larger teachers may be served through one slot, while gross transfer, post-rollout tail wait, and end-to-end GPU hours are separate required measurements.

1 INTRODUCTION

Post-training one student on several capabilities means choosing which teacher or reward service to call and deciding which task gradients enter an update. A task mixture often controls two decisions at once: how much each task counts in the objective and how often the system spends compute on it. Once that mixture follows loss gaps, gradient norms, or a curriculum, target choice and compute allocation become difficult to distinguish.

We separate these decisions. Objective weights λ define the weighted teacher-loss objective

$$F(\theta) = \sum_i \lambda_i L_i(\theta). \tag{1}$$

The execution policy separately decides which tasks supply the next expensive update. This distinction also keeps capability evaluation conceptually separate: one teacher loss can affect several downstream domains (Li et al., 2026). An experiment can now change allocation while keeping the teacher-loss objective fixed.

We treat execution granularity as a configuration rather than a premise. Each optimizer update includes K out of M tasks. The familiar task-at-a-time regime is $K = 1$, a standard mixed update is $K = M$, and intermediate values can trade statistical efficiency against systems cost. For every selected task we use one fixed unit of P prompts and G responses per prompt. Tokens are recorded but do not size the unit. This choice matches the per-response objective and prevents response length from being silently converted into a task weight.

The main objective normalizes each task by its warm-start teacher loss, so it measures average relative progress. This also makes the role of loss units explicit. If a raw loss is multiplied by a constant and its objective coefficient is divided by the same constant, both F and the allocation score $\lambda_i s_i$ stay unchanged. The reward-scale imbalance observed in Open-MOPD is therefore a reason to state and normalize the objective, rather than let implementation scale act as an implicit task weight (Gao et al., 2026).

Let π_i be the chance that task i is included. Weighting its gradient by λ_i/π_i gives the standard Horvitz–Thompson estimate of the fixed objective for every K. The resulting variance has two readable parts. One comes from omitting task means and shrinks as π_i approaches one; the other comes from noisy gradients within each task and remains even in a mixed batch. This decomposition turns the choice of execution regime into something that can be measured at the warm start.

GPAS uses each task's AdamW-scaled gradient magnitude to set the inclusion probabilities. For $K = 1$ this recovers sampling in proportion to $\lambda_i s_i$; for larger K, capped water filling increases inclusion until all tasks enter at $K = M$. A cost-aware version discounts tasks whose response unit takes longer on the measured training pipeline. The warm start also gives a useful headroom number: for $K = 1$ before the safety bound becomes active, the largest local reduction in the second-moment proxy over uniform sampling is $1 + \mathrm{CV}^2(\lambda_i s_i)$. If this number is close to one, the paper makes a mechanism claim rather than promising a large end-to-end gain.

AdamW introduces a second choice. Moment-consistent AdamW accumulates a weighted sum of taskwise squared gradients, whose population target is independent of K and the inclusion policy. At $K = M$ this is not the same as squaring the mixed gradient, as conventional AdamW does: the taskwise target is larger on coordinates where task gradients disagree. We therefore compare the two rules as optimizer configurations and do not assume that one must win.

Systems cost is equally configuration dependent. Our teachers are larger than the 1.7B student, but a mixed update need not keep all teachers resident: one teacher slot can score task groups sequentially. Conversely, $K = 1$ is not automatically faster, because switching that slot may still require model transfer. We prefetch the selected teacher when the hardware permits and measure both the tail wait after rollout and the realized end-to-end time. The decision among $K = 1$, an intermediate K, and $K = M$ is made from both this profile and the statistical headroom.

Our contributions are:

- A single fixed-objective estimator spanning $K = 1$ task-at-a-time updates, intermediate subsets, and $K = M$ mixed updates, together with a variance decomposition that prices what is gained by increasing K.
- GPAS and Cost-GPAS inclusion rules in the AdamW geometry, a simple warm-start headroom diagnostic, and bounded-age probes that make stale online estimates an accounted cost rather than an exploration assumption.
- A response-based MOPD protocol that measures optimizer noise, teacher transfer, peak memory, tokens, responses, and GPU hours under the same objective. Native token-weighted MOPD and D^3 are treated as different-objective references in the protocol.

2 Related Work

Objective weights and multi-task optimization. Multi-task methods often tune loss weights or edit gradient directions to change the capability trade-off (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018). Large empirical comparisons found carefully tuned scalarization competitive with specialized multi-task optimizers in their evaluated settings (Kurin et al., 2022; Xin et al., 2022). These results support our decision to state the weighted objective explicitly and evaluate execution under fixed weights. We normalize task losses at the common initial checkpoint so equal importance has a concrete per-response meaning. GPAS can also accept a time-varying objective supplied by a curriculum, but those results are labeled as a different target.

Adaptive data and prompt schedules for language models. Online Data Mixing uses a bandit to update pretraining-domain proportions (Albalak et al., 2023). Recent post-training methods schedule distributions from average advantage, skip low-value rollouts, or select prompts near the current learning frontier (Wang et al., 2025; Zheng et al., 2025; Gao et al., 2025). These methods optimize a domain or prompt policy with signals tied to learning progress. Our question concerns task-level teacher compute: given stated objective weights, which proposal gives an efficient unbiased gradient in the current optimizer geometry? The fixed-target comparisons place heuristic schedules and GPAS under the same objective.

Multi-teacher post-training. MOPD routes student rollouts to task teachers and trains a shared student with dense token feedback (Ma et al., 2026). Open-MOPD identifies token count, reward scale, and stale rewards as sources of unequal update budgets (Gao et al., 2026). D^3-MOPD uses teacher-loss gap and descent velocity to update the domain mixture (Sun et al., 2026). Teacher mixtures can also create cross-domain capability transfer and a mixture-dependent seesaw (Li et al., 2026). The cross-domain transfer results motivate our use of teacher loss for objective comparisons and separate benchmark reporting for capability. We distinguish the published task-at-a-time D^3 schedule from a genuinely mixed D^3 update; a fixed-objective version that uses the D^3 score only as a proposal is an ablation.

Adaptive importance sampling. Gradient-norm importance sampling reduces the second moment of an SGD estimate (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Cost-aware sampling adds a square-root cost correction and provides convergence guarantees beyond convex objectives (Mohri et al., 2026). The same allocation shape appears in classical Neyman allocation (Neyman, 1934; Cochran, 1977). Online importance samplers face partial feedback because only selected tasks receive fresh scale observations. Bandit formulations provide exploration and regret controls for this setting (Namkoong et al., 2017; Salehi et al., 2017; Borsos et al., 2018). Our bounded-age probes provide the freshness mechanism. A small inclusion floor is used only to bound importance multipliers.

Sampling with adaptive optimizers. Lahire showed that the gradient-norm proposal for SGD loses its optimality for RMSProp and Adam and argued that sampling should match the optimizer (Lahire, 2023). GPAS gives a constructive answer for AdamW: proposal-independent taskwise second-moment observations followed by an allocation rule in the resulting preconditioned geometry. Our contribution also spans exact-K subsets, making task-at-a-time and mixed updates endpoints of one execution choice.

3 A Fixed Objective across Update Configurations

There are M tasks. Let ℓ_i be task i's reverse-KL training surrogate, averaged first within each student response and then over responses and prompts. As in the implementation, its gradient treats the current rollout distribution as fixed. The main experiment uses the normalized loss $L_i = \ell_i/\ell_i(0)$, where the denominator is measured once at the common initial checkpoint, and sets $\lambda_i = 1/M$. At the objective and first-moment level this is equivalent to raw-loss coefficients proportional to $1/\ell_i(0)$; squared-gradient observations are formed only after normalization. More generally, positive weights with $\sum_i \lambda_i = 1$ define Equation 1.

One *task unit* contains P prompts and G independently sampled responses per prompt. Token losses are averaged within a response; the PG response gradients are then averaged to form G_i. Response length is capped and logged, but it does not change how many observations a task unit contains.

An update selects a set S of exactly K tasks. Task i has inclusion probability π_i, so $\sum_i \pi_i = K$ and $\pi_i \leq 1$. Every selected task produces one independent task unit. We use

$$\widehat{g} = \sum_{i \in S} \frac{\lambda_i}{\pi_i} G_i. \tag{2}$$

Lemma 1 (Standard importance correction)**.** *The corrected gradient has mean*

$$\mathbb{E}[\widehat{g}] = \sum_i \lambda_i \mathbb{E}[G_i] \equiv h. \tag{3}$$

This familiar Horvitz–Thompson result fixes the weighted gradient target for any K. For the stop-gradient rollout surrogate used here, h is the conditional gradient of Equation 1. For $K = 1$, π_i is the usual task-sampling probability. For $K = M$, every $\pi_i = 1$ and Equation 2 is the mixed gradient. A curriculum may change λ between updates; the same statement applies after each change.

The correction interacts with AdamW through its second moment. Define the taskwise square observation

$$\widehat{u} = \sum_{i \in S} \frac{\lambda_i}{\pi_i} G_i^2, \tag{4}$$

where the square is element-wise. Moment-consistent AdamW uses $\widehat{g}$ for its first moment and $\widehat{u}$ for its second:

$$\begin{aligned} m^+ &= \beta_1 m + (1-\beta_1)\widehat{g}, \\ v^+ &= \beta_2 v + (1-\beta_2)\widehat{u}. \end{aligned} \tag{5}$$

Bias correction, weight decay, and the parameter update stay unchanged.

Proposition 1 (Proposal-independent second-moment target)**.** *At a fixed training state, let $S_i = \mathbb{E}[G_i^2]$. The mean observation in the moment-consistent second-moment update is*

$$\sum_i \lambda_i S_i, \tag{6}$$

which is independent of K and the inclusion probabilities. Conventional AdamW instead observes $\widehat{g}^2$. At $K = 1$ its mean is $\sum_i \lambda_i^2 S_i / \pi_i$; at $K = M$ it is the square of the mixed gradient. Thus conventional AdamW changes its second-moment target across the spectrum.

Appendix B.1 gives the two-line calculation. The claim concerns the second-moment target at a fixed state. Task order can still produce different parameter paths, and the target moves as the model learns.

At the mixed endpoint, Jensen's inequality gives $\sum_i \lambda_i G_i^2 \geq (\sum_i \lambda_i G_i)^2$ coordinate-wise when the weights sum to one. The gap is the disagreement among task gradients. Moment consistency can therefore produce a larger denominator and a smaller effective learning rate on heterogeneous coordinates. Proposal independence is a clean comparison target, not a guarantee of better optimization; the experiment includes conventional mixed-batch AdamW as a necessary reference.

4 Pricing the Number of Tasks per Update

4.1 What increasing K removes

Let A denote AdamW's diagonal scaling before the current update. For task i, write $h_i = \mathbb{E}[G_i]$ and define its within-task noise after this scaling as

$$\sigma_i^2 = \mathbb{E}\|A(G_i - h_i)\|_2^2. \tag{7}$$

Let $h = \sum_i \lambda_i h_i$. In an independent-inclusion reference model, where the expected rather than realized task width is K, the error of Equation 2 has the decomposition

$$\mathbb{E}\|A(\widehat{g} - h)\|_2^2 = \sum_i \lambda_i^2 \left[\frac{1-\pi_i}{\pi_i} \|Ah_i\|_2^2 + \frac{\sigma_i^2}{\pi_i} \right]. \tag{8}$$

The first term is the cost of leaving task means out of an update. It is largest near $K = 1$ and disappears when $\pi_i = 1$. The second term is gradient noise inside a task unit and remains at $K = M$. Exact fixed-size sampling adds a pairwise correction determined by how the K-task subsets are formed; we give that expression in Appendix B.2 and measure it directly in the frozen-gradient bank. Equation 8 is the simple pricing model used to choose the inclusion marginals.

Define the total scaled magnitude

$$s_i^2 = \mathbb{E}\|AG_i\|_2^2 = \|Ah_i\|_2^2 + \sigma_i^2. \tag{9}$$

Up to a term that is fixed before the task draw, minimizing Equation 8 amounts to minimizing

$$V(\pi) = \sum_i \frac{\lambda_i^2 s_i^2}{\pi_i}, \quad \sum_i \pi_i = K, \quad \pi_{\min} \leq \pi_i \leq 1. \tag{10}$$

The solution is lower-and-upper capped water filling,

$$\pi_i = \min\left(1, \max\left(\pi_{\min}, \frac{\lambda_i s_i}{\tau}\right)\right), \tag{11}$$

where τ is chosen so the probabilities sum to K. A fixed-size sampler then draws exactly K tasks with these true marginals. With four tasks, we enumerate the possible K-subsets and choose the maximum-entropy distribution with those marginals, which also gives the pairwise inclusion probabilities used in evaluation. We call this rule Gradient-Preconditioned Adaptive Sampling (GPAS). We choose $\pi_{\min}$ so that $M\pi_{\min} \leq K$. At $K = 1$, when the lower bound is inactive, it reduces to ordinary sampling proportional to $\lambda_i s_i$; at $K = M$ every task is included.

The warm start reveals how much this allocation can matter before an end-to-end run. For $K = 1$ when the lower bound is inactive,

$$\frac{V(\text{uniform})}{V(\text{GPAS})} = 1 + \mathrm{CV}^2(\lambda_i s_i). \tag{12}$$

This is local headroom in the uncentered proxy, not a promised loss reduction. The protocol places it before the training curves. When all tasks are already included, there is no inclusion decision left. If the mixed batch has extra response units to distribute, the separate within-task quota problem uses group-level centered noise rather than the uncentered magnitude s_i; its short derivation is in Appendix B.4.

4.2 Runtime-aware allocation

Let c_i be the measured attributed time of one fixed-response task unit under the chosen sequential streaming policy. A simple additive compute proxy is

$$J(\pi) = \left(\sum_i \pi_i c_i\right) V(\pi). \tag{13}$$

When neither probability bound is active, minimizing this product gives the familiar square-root cost rule

$$\pi_i \propto \frac{\lambda_i s_i}{\sqrt{c_i}}. \tag{14}$$

When either bound is active, we solve the four-variable constrained problem directly rather than present a more elaborate closed form. We call the result Cost-GPAS.

The cost is not teacher FLOPs or raw model-load time. In our streamed-teacher implementation, task units are served sequentially, while transfer within a unit can overlap student rollout. For $K = 1$, c_i is the complete observed task-step time. For larger K, it is an additive attribution that excludes the common optimizer launch. Teacher reuse and overlap-induced contention make the cost history dependent, so c_i is only a local average. The realized GPU-hour curve, not Equation 13 alone, determines the cross-K systems conclusion.

4.3 Online estimates with bounded age

The current backward pass supplies one observation of s_i^2 for every selected task. GPAS stores its exponential average; Cost-GPAS also stores the task-unit time. A round-robin warm start initializes every task. Thereafter, if an estimate is older than $A_{\max}$ processed task units, an eight-response probe runs rollout, teacher scoring, and backward for that task but does not update the student. Its mean/noise decomposition is converted to the 64-response task-unit scale before updating s_i; its runtime is logged separately and never inserted into the task-unit cost average. Probe responses and GPU time count in every resource total.

Initialize one scale and cost estimate per task with a round-robin warm start. For each update: (1) refresh any estimate older than $A_{\max}$ with a counted, batch-size-corrected probe; (2) compute bounded inclusion probabilities for the chosen K and draw an exactly K-task set with those marginals; (3) run one fixed-response unit for each selected task and accumulate Equation 2; (4) take one AdamW update, using Equation 4 for the moment-consistent arm; and (5) update the selected tasks' scale and time averages. Save the sampler state with the optimizer.

Algorithm 1: GPAS for any task width K. The same procedure covers task-at-a-time and streamed mixed updates.

This age rule supplies exploration directly. The fixed lower bound in Equation 10 is retained only to bound the importance multiplier λ_i/π_i, not as the mechanism that discovers stale tasks. Every Horvitz–Thompson weight uses the final bounded marginal. At scheduled all-task checkpoints, fresh scales give the measured efficiency gap $J(\widehat{\pi})/J(\pi^{\text{fresh}})$ against the fresh separable solution and a common checkpoint cost vector; between checkpoints we log the corresponding value using the newest available reference. No sweep over floors or averaging constants is part of the main experiment.

5 EXPERIMENTS

The empirical protocol asks one joint question: after fixing a per-response teacher-loss objective, which task width and allocation rule gives the best progress per response and per GPU hour? The warm-start report must first show whether there is meaningful allocation headroom. Frozen-state measurements then test the variance and moment predictions, and the planned end-to-end MOPD runs provide the deciding comparison.

5.1 FOUR-TEACHER MOPD SETUP

The common student initialization is a fixed revision of Qwen3-1.7B (Qwen Team, 2025). Four fixed 4B/8B task specialists provide mathematics, code, instruction-following, and science feedback. The final manifest lists each exact checkpoint and holds the teacher set fixed across K. It also records precision, sharding, data revisions, prompt templates, hardware, and software versions.

One task unit draws a fixed count of 16 prompts from a frozen per-task stream and generates four student responses per prompt, for 64 attempted responses. Reverse KL is averaged over valid tokens within a response, over four responses within a prompt, and then over prompts. The response cap is 8,192 tokens, temperature is 1, and top-p is 1. Responses are never regenerated to fill a quota. Truncated responses contribute all available valid tokens; an empty generation or teacher-scoring failure contributes no update and receives a fixed preregistered penalty in reported loss. Every attempt remains in the response and resource counts.

At the common initial checkpoint, held-out prompts determine $\ell_i(0)$. The main objective is

$$F_{\text{rel}}(\theta) = \frac{1}{M} \sum_i \frac{\ell_i(\theta)}{\ell_i(0)}. \tag{15}$$

Thus "equal importance" means equal relative improvement, not equal raw loss units. The weights and denominators are frozen before method comparisons. A uniform average of unnormalized losses is evaluated on the frozen gradient bank as a scale-sensitivity check, not as a second suite of training runs.

All methods start from one common round-robin warm-start checkpoint. Each task contributes two task units to that checkpoint. Every arm then uses a fixed budget of 128,000 attempted responses, including warm-start and probe responses; different probe counts may therefore leave different numbers of optimization task units. Curves are compared on their common all-in response interval. The nth request for a task receives the same prompt group across methods.

Optimizer time is also placed on this response clock. Learning-rate warmup and decay are indexed by attempted responses. If β_1 and β_2 denote the one-task-unit AdamW decays, a K-task update uses

β_1^K and β_2^K; decoupled weight decay is converted in the same way to match cumulative decay per response. The remaining difference in update count is an intended part of the K configuration, not an accidental schedule change.

5.2 ONE TEACHER SLOT FOR EVERY K

We distinguish statistical task width from teacher residency. K is the number of task units combined before one optimizer update; the number of teachers resident in accelerator memory is a separate systems choice. Our fixed-resource implementation keeps at most one teacher resident for every K. A $K = 4$ update is therefore statistically mixed but physically streamed: the four task gradients are accumulated at one student parameter state while their teachers are visited one at a time.

The task set is chosen before generation. If the resident teacher is in the set, its task is served first; remaining tasks follow a fixed task-ID order. A switch evicts the resident teacher and loads and initializes the incoming teacher in the same slot, without holding two complete teacher weight sets. The runtime attempts this switch while the student produces the incoming task's responses, and teacher scoring waits for both. For each non-final task, eviction of its teacher, and loading of the next teacher when memory permits, may overlap student backward. The final teacher is retained. Gradients and their taskwise squares are accumulated online without storing all K task gradients, and one optimizer update follows the final task unit.

Every switch records offload start/end, load start/end, and teacher-ready time. We define transfer tail wait as $\max(0, t_{\text{ready}} - t_{\text{rollout end}})$. This quantity does not capture rollout or backward slowdown caused by bandwidth contention, so the net systems comparison uses end-to-end GPU hours. GPU hours equal wall time times all reserved GPUs, including periods spent waiting or transferring. The manifest fixes the number and type of GPUs across K and records which devices hold the student and teacher slot, the host-memory or storage staging source, the PCIe/NVLink path, and whether the teacher engine persists across swaps. We report the following quantities from the main runs:

Table 1: System measurements reported for each task width. A numeric version of this table is a submission requirement.

Category	Measurements
Residency	worker placement, parameters, precision and sharding, steady and transient peak HBM/GPU, teacher-slot and MC-accumulator memory
Teacher switching	switch/cache-hit rate per task unit, conditional raw switch latency, tail wait per task unit, teacher-late fraction
Critical path	rollout, scoring, backward, optimizer, p50/p95 K-update time and GPU-seconds per task unit
Throughput	responses/GPU-hour, tokens/GPU-hour, and total GPU hours

The HBM row records the student rollout KV working set, transfer buffers, and optimizer accumulators at the transient peak. If all-teacher residency exceeds the same measured budget, the one-slot policy is a feasibility constraint; otherwise it is simply the deployment evaluated here. A concentrated $K = 1$ proposal may reuse the resident teacher and realize fewer switches, while a larger K amortizes the optimizer launch. The traces determine whether either effect is material.

5.3 CORE MATCHED RUNS

The core study is deliberately small. It compares three allocation rules at the task-at-a-time endpoint and two optimizer definitions at the fully mixed endpoint. The framework supports intermediate widths, but $K = 2$ is outside the minimum study and would require a separately preregistered extension.

Table 2: Core fixed-objective MOPD comparisons. Each row is one matched run, not a multi-seed suite. "MC" denotes moment-consistent AdamW.

Configuration	K	Inclusion rule	AdamW second moment
Task-at-a-time Uniform	1	uniform	MC
Task-at-a-time GPAS	1	Eq. 11	MC
Task-at-a-time Cost-GPAS	1	Eq. 14	MC
Streamed mixed, taskwise moment	4	all tasks	MC
Streamed mixed, conventional	4	all tasks	mixed gradient squared

The warm-start bank also evaluates cost-only allocation ($\lambda_i/\sqrt{c_i}$), a response-length proxy, and GPAS without cost. These three rows use the same measured gradients and times and require no additional training run. Their held-out J values test whether gradient scale adds local information beyond cost or response length. The Cost-GPAS versus GPAS and Uniform runs separately test realized time efficiency; we do not claim an end-to-end comparison with proxy policies that were not trained.

Native token-weighted MOPD and D^3 are reference configurations because their objectives differ from Equation 15. We label the published task-at-a-time version "D^3-schedule ($K = 1$)" and a true mixed version "D^3 mixed ($K = 4$)." These references are not part of the minimum five-run study and are shown only when matching results already exist. D^3-Proposal-IC is reserved as an appendix ablation after the core study is complete.

5.4 Endpoints and uncertainty

The primary optimization outcome is mean relative teacher loss from Equation 15. The two primary panels plot it against cumulative attempted responses and GPU hours, counting warm-start and probe work on both axes. The outcome table reports the final mean and every task's relative loss, responses and GPU hours to a common loss threshold, peak HBM, and responses/GPU-hour. Tokens/GPU-hour is reported in the systems table, but token count is not the sampling unit or the objective weight.

Each configuration is run once from the shared checkpoint. End-to-end curves are therefore descriptive matched-run comparisons, not estimates of training-seed variation. Intervals on held-out loss use paired bootstrap resampling of the same evaluation prompts with fixed evaluation-generation seeds and are labeled as evaluation-set uncertainty. Mechanism intervals resample task units within each checkpoint; checkpoints are never pooled as exchangeable observations. No multi-seed experiment or standard error across training runs is reported.

Capability reporting includes MATH-500 greedy pass@1, a frozen post-cutoff LiveCodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4. These outcomes describe the final capability profile; they do not replace the declared teacher-loss objective.

5.5 Measured mechanism and freshness cost

Before adaptive training, the common warm start reports each task's preconditioned mean magnitude, within-task noise, task-unit time, and the headroom in Equation 12. When a probability bound is active, we also report $V(\text{uniform})/V(\pi^{\text{wf}})$ for the fresh bounded water-filled marginals. This is a separable oracle, not the exact fixed-size optimum. If the measured headroom is below a preregistered practical threshold, the end-to-end claim is limited to regime pricing rather than task-selection speedup; the task set is not replaced after inspecting training outcomes.

One designated Uniform reference trajectory supplies the warm, middle, and late checkpoints. At each saved state, an offline bank of eight independent task units per task estimates the exact fixed-K variance, the mean/noise decomposition in Equation 8, and the task-dispersion gap between the two AdamW second moments. Proposal quantities are estimated on one half of the bank and evaluated on the other, then the halves are swapped. Bank collection is charged and reported separately from the end-to-end training curves.

We fix $A_{\max} = 50$ processed optimization task units and use the eight-response probe described in Section 4. The probe estimate is converted to the 64-response scale from maintained prompt-group mean and noise components; its runtime never updates the main task-unit cost average. After a probe, the pre-probe teacher is restored, and both switches and all waiting time are charged. The next optimizer update therefore sees the same resident teacher it would have seen without the probe. At $K = 4$ every task is observed each update, so no stale-task probe is needed.

The logs contain age, probe count and cost, final bounded inclusion marginals, importance multipliers, and $J(\widehat{\pi})/J(\pi^{\text{fresh}})$ at fresh all-task checkpoints. This ratio uses one fixed checkpoint cost vector and the fresh separable Cost-GPAS solution. A fixed small probability bound is used only as a numerical safety limit. We do not sweep the bound, moving-average constant, or probe age.

Empirical completion gate. The current workspace contains the controlled implementation checks but not the MOPD checkpoints, traces, or loss tables required by this section. No end-to-end number is inferred from the theory. Before submission, the numeric system table, warm-start headroom, loss curves, and final MOPD outcome table must appear in the main paper; otherwise the empirical contribution and any efficiency claim are removed from the abstract and conclusion.

6 Limitations and Conclusion

The framework makes the execution choice falsifiable. If $K = M$ gives the best loss per GPU hour, task-at-a-time training is justified only by a deployment constraint, and the measured headroom quantifies its statistical cost. If $K = 1$ wins, the traces show whether teacher reuse and fewer realized switches repay that cost. A separately preregistered intermediate-K extension can use the same accounting when the endpoints expose a genuine trade-off.

The clean variance formula uses independent inclusion to price the marginals. Our implementation draws an exact fixed-size subset, whose variance also depends on pairwise inclusion and task-gradient alignment. This correction can have either sign for conflicting gradients. We therefore call water filling a separable allocation rule and evaluate the exact design at frozen checkpoints, rather than claim that first-order probabilities solve every fixed-size sampling problem.

Online estimates still lag a changing model. Bounded-age probes prevent unobserved tasks from remaining stale indefinitely and put a visible GPU-hour price on freshness. The small probability bound serves only as an importance- weight safeguard. The reported $J(\widehat{\pi})/J(\pi^{\text{fresh}})$ uses a fresh separable reference and a common checkpoint cost vector; between checkpoints it is an online diagnostic, not an exact fixed-size oracle.

Moment consistency is likewise a controlled design choice, not a universal improvement. At the mixed endpoint it accumulates taskwise squares, whereas conventional AdamW squares the mixed gradient. The difference is task dispersion and can reduce effective step sizes on heterogeneous coordinates. Reporting both mixed-batch arms establishes whether proposal-independent moments help or merely over-precondition this workload.

The runtime proxy treats task-unit costs as locally additive. With one resident teacher, the actual cost also depends on the previous teacher and on how much transfer overlaps rollout or backward. Cost-GPAS absorbs this mixture through recent measured times but is not a global scheduling optimum. The required traces include cache hits, switch rates, raw transfer, and post-rollout tail wait; every systems conclusion must use realized GPU hours. Larger teachers establish neither infeasibility nor a speed benefit without the accompanying memory and timeline measurements.

The central recommendation is simple: define the per-response objective first, choose the number of tasks per update from measured statistical and systems costs, and allocate within that choice using objective-weighted gradient scale. The framework leaves room for task-at-a-time, streamed mixed, or an intermediate configuration; the endpoint study makes the first choice without treating the initial implementation as the answer.

AI use statement

We used generative AI tools for language editing and manuscript organization. The authors reviewed every AI-assisted revision and take responsibility for the final text, claims, proofs, code, and experimental results.

ETHICS STATEMENT

The study evaluates optimization methods for existing language models with established reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect the reported capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation of the results.

REPRODUCIBILITY STATEMENT

Section 5 specifies the fixed prompt–response unit, relative-loss objective, exact-K sampling protocol, teacher-streaming policy, resource accounting, and matched evaluation streams. The appendix gives the estimator and fixed-size variance derivations. The current workspace contains the controlled implementation checks, but not the MOPD checkpoints, gradient bank, system traces, or outcome tables. Accordingly, this draft makes no numerical large-model efficiency claim. A submission version must release or archive a run manifest, exact model and tokenizer revisions, normalized-loss denominators, per-task prompt streams, optimizer and sampler states, inclusion and probe logs, teacher-transfer traces, per-prompt evaluation output, and the data behind every main-text MOPD figure and table.

REFERENCES

Guillaume Alain, Alex Lamb, Chinnadhurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL `https://arxiv.org/abs/1511.06481`.

Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL `https://arxiv.org/abs/2312.02406`.

Zalan Borsos, Andreas Krause, and Kfir Y. Levy. Online variance reduction for stochastic optimization. In *Proceedings of the 31st Conference on Learning Theory*, pp. 324–357, 2018. URL `https://proceedings.mlr.press/v75/borsos18a.html`.

Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL `https://proceedings.mlr.press/v80/chen18a.html`.

William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.

Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL `https://arxiv.org/abs/2608.19098`.

Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient LLM post-training, 2025. URL `https://arxiv.org/abs/2510.01135`.

Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL `https://proceedings.mlr.press/v80/katharopoulos18a.html`.

Vitaly Kurin, Alessandro De Palma, Ilya Kostrikov, Shimon Whiteson, and M. Pawan Kumar. In defense of the unitary scalarization for deep multi-task learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL `https://opt-ml.org/papers/2023/paper20.pdf`.

Zhaoyi Li, Deyang Kong, Yuan Wei, Evan Yang, Ranran Shen, Mahardika Krisna Ihsani, Ming Yang, Wei Zhang, Chuan Hao, Jian Yang, Ran Tao, Bryan Dai, Shikun Zhang, Wei Ye, Ying Wei, and Defu Lian. Every coin has two sides: On the dual nature of generalization in on-policy distillation of large language models, 2026. URL `https://arxiv.org/abs/2608.16647`.

Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL `https://arxiv.org/abs/2110.14048`.

Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL `https://arxiv.org/abs/2606.30406`.

Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL `https://arxiv.org/abs/2604.28020`.

Hongseok Namkoong, Aman Sinha, Steve Yadlowsky, and John C. Duchi. Adaptive sampling probabilities for non-smooth optimization. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 2574–2583, 2017. URL `https://proceedings.mlr.press/v70/namkoong17a.html`.

Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.

Qwen Team. Qwen3 technical report, 2025. URL `https://arxiv.org/abs/2505.09388`.

Farnood Salehi, L. Elisa Celis, and Patrick Thiran. Stochastic optimization with bandit sampling, 2017. URL `https://arxiv.org/abs/1708.02544`.

Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL `https://arxiv.org/abs/1810.04650`.

Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D^3-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL `https://arxiv.org/abs/2608.24987`.

Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. DUMP: Automated distribution-level curriculum learning for RL-based LLM post-training, 2025. URL `https://arxiv.org/abs/2504.09710`.

Derrick Xin, Behrooz Ghorbani, Justin Gilmer, Ankush Garg, and Orhan Firat. Do current multi-task optimization methods in deep learning even help? In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL `https://arxiv.org/abs/2001.06782`.

Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts, 2025. URL `https://arxiv.org/abs/2506.02177`.

A Practical Training Recipe

1. Declare the counted objective. Define one task unit as a fixed number of prompts and responses, and average tokens within each response. Measure each raw task loss once at the shared initial checkpoint and normalize it before constructing gradients. Record the objective separately from the execution policy.

2. Profile before choosing K. Run a common round-robin warm start. For every task, estimate the AdamW-scaled gradient magnitude, within-task noise, and critical-path time. Report the uniform-to-GPAS headroom and the memory/transfer profile before the training curves. Choose the endpoint comparison in advance; an intermediate K belongs to a separately preregistered extension of the minimum endpoint study.

3. Draw an exact-K task set. Compute the GPAS water-filled marginals, or solve the small constrained Cost-GPAS problem, including both probability bounds. Then draw exactly K tasks from a fixed-size design with those final marginals; for four tasks the subset distribution is enumerated explicitly. The first-moment accumulator adds $\lambda_i G_i/\pi_i$ as each task unit arrives. The moment-consistent second-moment accumulator separately adds $\lambda_i G_i^2/\pi_i$; it never squares the final mixed accumulator.

4. Stream teachers and measure switching cost. Keep the same teacher-residency policy for every K. With one slot, serve a resident teacher first and use a fixed order for the rest. Prefetch the needed teacher during student rollout when possible, retain the final teacher, and update the student only after all selected task gradients have accumulated. Record raw switch phases, post-rollout tail wait, and end-to-end time.

5. Bound freshness and save state. When a task's scale age reaches $A_{\max}$ processed task units, run a small no-update probe, convert its mean/noise estimate to the full task-unit scale, and restore the pre-probe teacher. Charge all probe responses, switches, and GPU time, but keep probe runtime out of the task-unit cost average. Save the final bounded marginals, selected set and order, probe state, transfer phases, response and token counts, gradient and clipping statistics, and the two optimizer accumulators. A resume test should reproduce the next task set and optimizer update exactly.

B Derivations and Evaluation Details

B.1 Horvitz–Thompson and moment targets

Let Z_i indicate whether task i is in the selected set. The set is chosen before drawing task-unit data, so Z_i and the potential batch gradient G_i are conditionally independent. Since $\mathbb{E}[Z_i] = \pi_i$, conditioning on the state before selection gives

$$\mathbb{E}[\widehat{g}] = \sum_i \mathbb{E}[Z_i] \frac{\lambda_i}{\pi_i} \mathbb{E}[G_i] = \sum_i \lambda_i \mathbb{E}[G_i]. \tag{16}$$

The same calculation with an element-wise square gives

$$\mathbb{E}[\widehat{u}] = \mathbb{E}\left[\sum_i Z_i \frac{\lambda_i}{\pi_i} G_i^2\right] = \sum_i \lambda_i \mathbb{E}[G_i^2]. \tag{17}$$

Neither result depends on K, the marginal probabilities, or the particular fixed-size design used to realize them.

For $K = M$ and weights summing to one, let $\bar{G} = \sum_i \lambda_i G_i$. The samplewise difference between the taskwise and conventional mixed-batch square observations is

$$\sum_i \lambda_i G_i^2 - \bar{G}^2 = \sum_i \lambda_i (G_i - \bar{G})^2 \geq 0 \tag{18}$$

coordinate-wise. This is weighted task dispersion. Momentum, ϵ, and the history of the exponential average still determine the realized update, so Equation 18 is not an optimization ordering.

B.2 EXACT FIXED-SIZE VARIANCE

Let π_{ij} be the probability that tasks i and j both enter the K-task set. Assuming independent task-unit noise across tasks, the exact fixed-size counterpart of Equation 8 is

$$\mathbb{E}\|A(\widehat{g}-h)\|_2^2 = \sum_i \lambda_i^2 \left[\frac{1-\pi_i}{\pi_i}\|Ah_i\|_2^2 + \frac{\sigma_i^2}{\pi_i}\right] + 2\sum_{i<j}\lambda_i\lambda_j\left(\frac{\pi_{ij}}{\pi_i\pi_j}-1\right)\langle Ah_i, Ah_j\rangle. \tag{19}$$

The pairwise inclusion covariance is often negative, but the final correction need not be: conflicting task means have a negative inner product. First-order marginals alone therefore do not determine the exact fixed-size optimum. GPAS optimizes the separable part and the experiment evaluates the complete Equation 19. For four tasks we enumerate all K-subsets and solve for the maximum-entropy distribution with the GPAS marginals, so π_{ij} is known exactly. At $K=1$ the pairwise term is fixed with respect to the proposal, and at $K=M$ all selection terms vanish.

B.3 WATER FILLING AND THE DESCENT CONNECTION

For positive $a_i = \lambda_i s_i$, consider $V(\pi) = \sum_i a_i^2/\pi_i$ on the lower-and-upper capped simplex. The interior optimality condition gives $\pi_i = a_i/\tau$. Coordinates outside the allowed interval are fixed at the nearest bound and the condition is repeated on the remaining tasks, which yields Equation 11.

For $K=1$ with the lower bound inactive, Cauchy–Schwarz gives

$$V(\pi) \geq \left(\sum_i \lambda_i s_i\right)^2, \tag{20}$$

with equality at GPAS. Uniform sampling gives $M\sum_i(\lambda_i s_i)^2$. Dividing these quantities produces Equation 12. For larger K the same ratio applies while no cap is active; after saturation it is an upper bound and the paper reports the directly calculated ratio.

In the independent-inclusion reference model and for a fixed preconditioner, the standard smoothness inequality for $\theta^+ = \theta - \eta A\widehat{g}$ contains a proposal-independent descent term and a second-order term proportional to $\mathbb{E}\|A\widehat{g}\|^2$. This makes V a local separable allocation criterion. The exact-K update also contains the pairwise term in Equation 19; the argument does not claim identical trajectories or a global fixed-size optimum.

B.4 COST RULE AND MIXED-BATCH QUOTA

Write $C(\pi) = \sum_i \pi_i c_i$. When neither probability bound is active, Cauchy–Schwarz gives

$$J(\pi) = C(\pi)V(\pi) \geq \left(\sum_i \lambda_i s_i \sqrt{c_i}\right)^2, \tag{21}$$

with equality when $\pi_i \propto \lambda_i s_i/\sqrt{c_i}$. If normalizing this ray to sum to K would violate either bound, the constrained product is solved numerically. Simply clipping the square-root scores is not generally exact.

At $K=M$, inclusion probabilities are fixed at one. Suppose instead that a mixed update can distribute N independent prompt groups across tasks, with n_i groups for task i. Let ξ_i^2 be the preconditioned noise of one group, including its four correlated responses, and let d_i be its cost. The remaining stochastic term is $\sum_i \lambda_i^2\xi_i^2/n_i$. Without cost differences, the usual Neyman quota is $n_i \propto \lambda_i\xi_i$; under the same variance–cost product used above it becomes

$$n_i \propto \frac{\lambda_i \xi_i}{\sqrt{d_i}}. \tag{22}$$

This is a separate within-task quota problem and uses centered noise rather than the uncentered GPAS scale. Correlated responses from one prompt are kept together when estimating ξ_i.

B.5 Loss units and the MOPD gradient

If a raw task loss is rescaled as $\ell_i' = b_i \ell_i$ and its coefficient as $\alpha_i' = \alpha_i / b_i$, then both the objective contribution and the GPAS score $\alpha_i s_i$ are unchanged. This explains why explicit objective coefficients prevent arbitrary reward or loss units from silently changing allocation. The taskwise second-moment target is not invariant to this re-expression. We therefore define the normalized loss $L_i = \ell_i / \ell_i(0)$ before forming both gradients and squared-gradient observations, rather than hiding normalization inside an optimizer weight.

For completeness, fix a student-generated prefix z, write $p_v = \pi_\theta(v \mid z)$ and $t_v = T_i(v \mid z)$. The reverse-KL contribution at that prefix is estimated by the sampled-token score

$$\mathrm{sg}\left[\log \frac{p_V}{t_V}\right] \nabla_\theta \log p_V, \qquad V \sim p. \tag{23}$$

The implementation averages these contributions within a response and then over the fixed prompt–response unit. The allocation results require only that G_i have a well-defined conditional task mean. Accordingly, Equation 1 is the local surrogate that holds the rollout distribution fixed during an update; we do not claim that this score differentiates through the complete on-policy rollout distribution.

B.6 Frozen-state evaluation

At each selected checkpoint, the gradient bank estimates task means, within-task noise, preconditioned magnitudes, and critical-path costs. One fold constructs the allocation and the other evaluates it; swapping and averaging uses all task units without evaluating a proposal on the same observations that created it. For $M = 4$, the sampler's enumerated subset distribution gives both π_i and π_{ij}, allowing direct evaluation of Equation 19. The same stored task gradients compare the conventional and moment-consistent second-moment observations.

The main unit averages 16 prompt groups, whereas an eight-response probe contains two groups. The sampler maintains a task mean and group-noise estimate and reconstructs the full-unit scale as $s_i^2 = \|Ah_i\|^2 + \xi_i^2/16$. A probe refreshes these components rather than inserting its raw squared norm into the 64-response average. Probe time is tracked separately from c_i.

For an interior cost optimum, freeze one common checkpoint cost vector and define $\widetilde{\pi}_i = \pi_i c_i / C(\pi)$. Direct substitution gives the diagnostic identity

$$\frac{J(\widehat{\pi})}{J(\pi^{\mathrm{fresh}})} = 1 + D_{\chi^2}(\widetilde{\pi}^{\mathrm{fresh}} \,\|\, \widetilde{\widehat{\pi}}). \tag{24}$$

We use it only when neither probability bound is active and both proposals use that same cost vector. Otherwise the ratio is calculated directly. Fresh all-task measurements give the separable reference at scheduled checkpoints; values between them use the newest reference and are labeled as online estimates rather than an exact fixed-size oracle.

B.7 Reference curricula, state, and system logs

Native MOPD and D^3 use their published token-weighted objectives. The task-at-a-time curriculum is labeled D^3-schedule ($K = 1$). A true mixed reference applies its current D^3 weights to all four task gradients before one update. D^3-Proposal-IC instead fixes the relative-loss objective, uses the published D^3 signal only to construct inclusion probabilities, and applies Equation 2. None is part of the minimum five-run study; the proposal variant is an appendix ablation only after core results exist.

Every checkpoint stores K, inclusion marginals, selected set and order, scale and cost averages, observation ages, probe counts, and the AdamW variant. Each task unit logs prompt, response, and token counts; gradient norms, clipping, and overflow; residency and cache states; transfer and teacher-ready times; rollout, scoring, backward, and optimizer times; GPU count; and peak HBM. Probe records cover both restore switches and preserve the next update's residency. The streamed taskwise-square accumulator reports its memory. Resume tests must reproduce the next allocation and optimizer update.